



UNIVERSIDAD NACIONAL JORGE BASADRE GROHMANN

FACULTAD DE INGENIERÍA
CICLO 6

ESCUELA PROFESIONAL DE INGENIERÍA EN INFORMÁTICA Y
SISTEMAS

“SIMULACIÓN DE GESTIÓN DE MEMORIA EN C++”

INFORME 06

SISTEMAS OPERATIVOS

PRESENTA

ALEX JEANPIER CHATA CHINO
2024-119012

07 de octubre del 2026

1. Introducción

La gestión de memoria es una función crítica del sistema operativo, responsable de coordinar y controlar el uso de la memoria principal. Su propósito es asignar y liberar bloques de memoria para los procesos en ejecución, optimizando el rendimiento general del sistema y garantizando que los programas cuenten con el espacio necesario para operar.

Dentro de esta gestión, la asignación dinámica de memoria, mediante técnicas de particionamiento dinámico, permite que el sistema operativo otorgue bloques de memoria de tamaño variable según las necesidades específicas de cada proceso en el momento de su ejecución. Esto difiere del particionamiento fijo, donde los bloques tienen tamaños predefinidos.

Debido a la naturaleza dinámica de las solicitudes y liberaciones de memoria, existen diferentes estrategias algorítmicas para decidir en qué bloque libre (o "hueco") se debe cargar un proceso. Algoritmos clásicos como First-Fit (Primer Ajuste), Best-Fit (Mejor Ajuste) y Next-Fit (Próximo Ajuste) buscan equilibrar la velocidad de asignación con la eficiencia en el uso del espacio, cada uno con sus propias ventajas y limitaciones frente al problema de la fragmentación.

2. Objetivos

Objetivo General

Analizar y comprender el comportamiento de los algoritmos de asignación dinámica de memoria (First-Fit, Best-Fit y Next-Fit) mediante su implementación y simulación en C++.

Objetivos Específicos

- Implementar y ejecutar los algoritmos First-Fit, Best-Fit y Next-Fit en C++.
- Comparar el comportamiento de los tres algoritmos ante los mismos tamaños de bloques de memoria y procesos.
- Identificar las ventajas, desventajas y limitaciones prácticas de cada estrategia de asignación observada.
- Responder a las cuestiones técnicas planteadas en la guía de la práctica.

3. Metodología

El desarrollo de la práctica se llevó a cabo simulando las estrategias de asignación mediante programas en C++. Inicialmente, se transcribió el código base del algoritmo First-Fit utilizando el editor de texto nano, guardándolo como first.cpp. Posteriormente, se compiló el código fuente y se ejecutó para observar los resultados de la asignación.

A partir de la estructura del primer programa, se procedió a implementar las variaciones correspondientes para los algoritmos Best-Fit (best.cpp) y Next-Fit (next.cpp), basándose en la lógica algorítmica y las fuentes proporcionadas. Para asegurar una comparación equitativa, los tres algoritmos se sometieron a prueba utilizando los mismos parámetros de entrada:

Tamaños iniciales de bloques de memoria	{100, 500, 200, 300, 600}
Tamaños de los procesos solicitantes	{212, 417, 112, 301}

Los resultados de la asignación para cada algoritmo fueron registrados y se analizan en la siguiente sección, apoyándose en las evidencias de compilación y ejecución obtenidas en la terminal.

4. Análisis de Resultados

4.1. Análisis de First-Fit

El algoritmo First-Fit asigna el primer bloque de memoria disponible que sea lo suficientemente grande para alojar al proceso.

```

1  #include <iostream>
2  using namespace std;
3
4  void firstFit(int blockSize[], int m, const int processSize[], int n)
5  {
6      // Arreglo dinámico para guardar el bloque asignado a cada proceso (-1 si no se asigna)
7      int* allocation = new int[n];
8
9      for (int i = 0; i < n; ++i)
10         allocation[i] = -1;
11
12     // Algoritmo First-Fit
13     for (int i = 0; i < n; ++i) {
14         for (int j = 0; j < m; ++j) {
15             if (blockSize[j] >= processSize[i]) {
16                 allocation[i] = j;           // asigna bloque j al proceso i
17                 blockSize[j] -= processSize[i]; // reduce espacio libre del bloque
18                 break;                       // pasa al siguiente proceso
19             }
20         }
21     }
22
23     // Salida
24     cout << "\nNo. Proceso\tTamano Proceso\tNo. Bloque\n";
25     for (int i = 0; i < n; ++i) {
26         cout << " " << (i + 1) << "\t\t" << processSize[i] << "\t\t";
27         if (allocation[i] != -1)
28             cout << (allocation[i] + 1);
29         else
30             cout << "No Asignado";
31         cout << "\n";
32     }
33
34     delete[] allocation; // libera memoria
35 }
36
37 int main()
38 {
39     int blockSize[] = {100, 500, 200, 300, 600};
40     int processSize[] = {212, 417, 112, 301};
41
42     int m = sizeof(blockSize) / sizeof(blockSize[0]);
43     int n = sizeof(processSize) / sizeof(processSize[0]);
44
45     firstFit(blockSize, m, processSize, n);
46     return 0;
47 }

```

Figura 1. Código del algoritmo First-Fit.

El análisis paso a paso de la ejecución revela el siguiente comportamiento:

- Estado inicial de bloques: {100, 500, 200, 300, 600}
- Proceso 1 (Tamaño 212): Busca desde el inicio. El bloque 1 (100) es muy pequeño. El bloque 2 (500) es suficiente. Se le asigna. El bloque 2 se reduce a $500 - 212 = 288$. Estado de bloques: {100, 288, 200, 300, 600}
- Proceso 2 (Tamaño 417): Busca desde el inicio. Los bloques 1, 2, 3 y 4 son muy pequeños. El bloque 5 (600) es suficiente. Se le asigna. El bloque 5 se reduce a $600 - 417 = 183$. Estado de bloques: {100, 288, 200, 300, 183}
- Proceso 3 (Tamaño 112): Busca desde el inicio. El bloque 1 (100) es muy pequeño. El bloque 2 (ahora de 288) es suficiente. Se le asigna. El bloque 2 se reduce a $288 - 112 = 176$. Estado de bloques: {100, 176, 200, 300, 183}
- Proceso 4 (Tamaño 301): Busca desde el inicio. Compara con los bloques actuales (100, 176, 200, 300, 183). Ningún bloque es lo suficientemente grande para alojarlo.

¿Qué sucede con el proceso 4 definido en processSize con tamaño 301? ¿A qué se debe ese comportamiento?

El proceso 4, que requiere un tamaño de 301, no es asignado a ningún bloque. Esto se debe a que, al aplicar la lógica de First-Fit con particionamiento dinámico, las asignaciones previas han fragmentado los bloques originales. Aunque la suma total de memoria libre es de 959 ($100+176+200+300+183$), esta memoria está fragmentada en bloques pequeños, y el bloque contiguo más grande disponible es de tamaño 300, el cual es insuficiente para satisfacer la solicitud de 301. Esto ilustra el problema de la fragmentación externa.

```
jeanpier@jeanpiernet:~/Documents/TEMP/04_memoria/src$ g++ first.cpp -o first.exe
jeanpier@jeanpiernet:~/Documents/TEMP/04_memoria/src$ ./first.exe
```

No. Proceso	Tamano Proceso	No. Bloque
1	212	2
2	417	5
3	112	2
4	301	No Asignado

Figura 2. Ejecución del algoritmo First-Fit.

4.2. Diferencia entre particionamiento dinámico y fijo

¿Que pasaría si se aplicara el mismo algoritmo, pero a bloques del mismo tamaño vistos en particionamiento fijo de memoria?

Si el algoritmo First-Fit se aplicara a bloques de memoria de tamaño fijo (particionamiento fijo), el comportamiento cambiaría significativamente. En el particionamiento dinámico, cuando a un proceso de tamaño X se le asigna un bloque de tamaño Y (donde $Y > X$), el tamaño del bloque libre se reduce a $Y - X$, quedando ese fragmento sobrante disponible para futuros procesos.

En el particionamiento fijo, por el contrario, un bloque entero se asigna a un solo proceso. Si al proceso de 212 se le asigna el bloque de 500, el espacio sobrante de 288 no se reduce para ofrecer un nuevo bloque libre, sino que permanece inútil y "desperdiciado" dentro de esa partición mientras el proceso 1 esté en ejecución. Este fenómeno se conoce como fragmentación interna. En el ejemplo dado, esto significaría que el proceso 3 (tamaño 112) no podría acomodarse en el sobrante del bloque de 500, y tendría que asignarse al bloque 3 de 200, alterando por completo el patrón de asignaciones subsiguientes.

4.3. Análisis de Best-Fit

El algoritmo Best-Fit busca minimizar el espacio desperdiciado buscando en todos los bloques disponibles y seleccionando aquel que sea lo suficientemente grande, pero que deje el menor espacio sobrante posible (el ajuste más ajustado).

Al analizar el código implementado para Best-Fit, es crucial realizar una observación técnica: la implementación proporcionada no realiza un verdadero particionamiento dinámico en la gestión de los tamaños de los bloques. En lugar de reducir el tamaño del bloque libre restando el tamaño del proceso (como sí hace First-Fit), el código utiliza la instrucción `blockSize[bestIdx] = -1`. Esto significa que una vez que se asigna un bloque a un proceso, ese bloque queda completamente invalidado para futuras búsquedas, simulando más bien un comportamiento de particionamiento fijo, donde un bloque solo puede ser ocupado por un único proceso.

```

1  #include <iostream>
2  using namespace std;
3
4  void bestFit(int blockSize[], int m, int processSize[], int n) {
5      int allocation[n];
6      int originalBlockSize[m];
7
8      // Cargas el tamaño de bloques original antes de la modificación
9      for (int i = 0; i < m; i++)
10         originalBlockSize[i] = blockSize[i];
11
12     // Inicializar el array allocation
13     for (int i = 0; i < n; i++)
14         allocation[i] = -1;
15
16     // Best Fit Allocation
17     for (int i = 0; i < n; i++) {
18         int bestIdx = -1;
19         for (int j = 0; j < m; j++) {
20             if (blockSize[j] >= processSize[i]) {
21                 if (bestIdx == -1 || blockSize[bestIdx] > blockSize[j])
22                     bestIdx = j;
23             }
24         }
25
26         // Si allocate ha sido encontrado
27         if (bestIdx != -1) {
28             allocation[i] = bestIdx;
29             blockSize[bestIdx] = -1; // Marcar el bloque como usado
30         }
31     }
32
33     // Imprimir resultados
34     cout << "No. Proceso\tTamaño de proceso\tNo. Bloque Asignado" << endl;
35     for (int i = 0; i < n; i++) {
36         cout << " " << i + 1 << "\t\t\t" << processSize[i] << "\t\t\t";
37         if (allocation[i] != -1)
38             cout << (allocation[i] + 1);
39         else
40             cout << "No asignado";
41         cout << endl;
42     }
43 }
44
45 int main() {
46     int blockSize[] = {100, 500, 200, 300, 600};
47     int processSize[] = {212, 417, 112, 301};
48     int m = sizeof(blockSize) / sizeof(blockSize[0]);
49     int n = sizeof(processSize) / sizeof(processSize[0]);
50
51     bestFit(blockSize, m, processSize, n);
52     return 0;
53 }

```

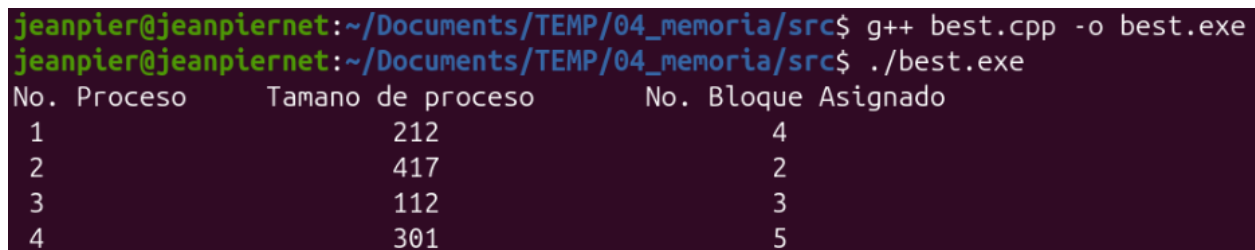
Jeanpier-CRH, 9 hours ago • feat: implementar el programa del mejor ajuste

Figura 3. Código del algoritmo Best-Fit.

Analizando la ejecución basada en esta implementación:

- Proceso 1 (212): Bloques aptos (500, 300, 600). El mejor ajuste es el bloque 4 (300). El bloque 4 se marca como no disponible (-1).
- Proceso 2 (417): Bloques aptos restantes (500, 600). El mejor ajuste es el bloque 2 (500). El bloque 2 se marca como no disponible (-1).
- Proceso 3 (112): Bloques aptos restantes (200, 600). El mejor ajuste es el bloque 3 (200). El bloque 3 se marca como no disponible (-1).
- Proceso 4 (301): Bloques aptos restantes (600). El mejor ajuste es el bloque 5 (600). Se le asigna.

Como se puede observar, bajo esta implementación simplificada, el Proceso 4 sí es asignado (al bloque 5), a diferencia de First-Fit. Sin embargo, esto es un subproducto de que Best-Fit eligió bloques más ajustados inicialmente (como el de 300 para el proceso de 212, dejando libre el bloque de 600), y de la naturaleza de partición fija que implica marcar el bloque completo como -1.



```
jeanpier@jeanpiernet:~/Documents/TEMP/04_memoria/src$ g++ best.cpp -o best.exe
jeanpier@jeanpiernet:~/Documents/TEMP/04_memoria/src$ ./best.exe
```

No. Proceso	Tamano de proceso	No. Bloque Asignado
1	212	4
2	417	2
3	112	3
4	301	5

Figura 4. Resultado de la implementación de Best-Fit.

4.4. Análisis de Next-Fit

El algoritmo Next-Fit (Próximo Ajuste) es una modificación de First-Fit. Su característica principal es que no comienza la búsqueda desde el primer bloque de memoria para cada proceso. En su lugar, mantiene un puntero (la variable j en el código) a la posición del último bloque asignado, y la búsqueda para el siguiente proceso comienza a partir de ahí. Si llega al final de la memoria, retorna al principio (implementado en el código como: *if* ($j == m$) $j = 0$;).

```

1  #include <iostream>
2  using namespace std;
3
4  void nextFit(int blockSize[], int m, int processSize[], int n) {
5      int allocation[n];
6      for (int i = 0; i < n; i++)
7          allocation[i] = -1;
8
9      int j = 0;
10     for (int i = 0; i < n; i++) {
11         while (j < m) {
12             if (blockSize[j] >= processSize[i]) {
13                 allocation[i] = j;
14                 blockSize[j] -= processSize[i];
15                 break;
16             }
17             j++;
18         }
19         if (j == m)
20             j = 0;
21     }
22
23     cout << "No. Proceso\tTamano de proceso\tNo. Bloque Asignado" << endl;
24     for (int i = 0; i < n; i++) {
25         cout << " " << i + 1 << "\t\t\t" << processSize[i] << "\t\t\t";
26         if (allocation[i] != -1)
27             cout << allocation[i] + 1;
28         else
29             cout << "No asignado";
30         cout << endl;
31     }
32 }
33
34 int main() {
35     int blockSize[] = {100, 500, 200, 300, 600};
36     int processSize[] = {212, 417, 112, 301};
37     int m = sizeof(blockSize) / sizeof(blockSize[0]);
38     int n = sizeof(processSize) / sizeof(processSize[0]);
39     nextFit(blockSize, m, processSize, n);
40     return 0;
41 }

```

Figura 5. Código del algoritmo Next-Fit.

Análisis de la ejecución (con particionamiento dinámico, ya que el código usa *blockSize[j] -= processSize[i]*):

- Proceso 1 (212): Inicia en índice 0. Asigna el bloque 2 (500). El bloque 2 se reduce a 288. El índice de búsqueda se queda en el bloque 2.
- Proceso 2 (417): Inicia la búsqueda desde el bloque 2 (ahora de 288, insuficiente). Sigue al bloque 3 (200), bloque 4 (300) y finalmente asigna el bloque 5 (600). El bloque 5 se

reduce a 183. El índice de búsqueda se queda en el bloque 5.

- Proceso 3 (112): Inicia búsqueda desde el bloque 5. Al llegar al final, vuelve al inicio. El bloque 1 (100) es insuficiente. El bloque 2 (288) es suficiente. Se le asigna. El bloque 2 se reduce a 176. El índice se queda en el bloque 2.
- Proceso 4 (301): Inicia desde el bloque 2 (176, insuficiente). Sigue al bloque 3 (200), bloque 4 (300), y bloque 5 (183). Ninguno es suficiente. No es asignado.

Al igual que First-Fit, el Proceso 4 no es asignado debido a la fragmentación externa. Next-Fit tiende a distribuir los bloques de manera más uniforme por toda la memoria, reduciendo la concentración de bloques pequeños al principio, lo cual es una ventaja sobre First-Fit en términos de velocidad de búsqueda, aunque puede experimentar fragmentación similar.

```
jeanpier@jeanpiernet:~/Documents/TEMP/04_memoria/src$ g++ next.cpp -o next.exe
jeanpier@jeanpiernet:~/Documents/TEMP/04_memoria/src$ ./next.exe

No. Proceso      Tamano de proceso      No. Bloque Asignado
1                212                    2
2                417                    5
3                112                    2
4                301                    No asignado
```

Figura 6. Resultado de la ejecución de Next-Fit. .

4.5. Comparación general

Algoritmo	Criterio de selección	Ventaja principal	Desventaja principal
First-Fit	El primer bloque libre lo suficientemente grande, buscando desde el inicio.	Simple de implementar y rápido en encontrar una asignación.	Tiende a acumular bloques pequeños y fragmentados al inicio de la memoria.
Best-Fit	El bloque libre más pequeño que sea suficiente, buscando en toda la memoria.	Minimiza el espacio desperdiciado en la partición elegida (menor fragmentación interna).	Más lento (requiere buscar en toda la lista). Tiende a generar huecos libres muy pequeños, aumentando la fragmentación externa.

Next-Fit	El primer bloque libre suficiente, buscando desde la última posición asignada.	Distribuye las asignaciones uniformemente. Suele ser más rápido que First-Fit al evitar reescanear el inicio.	Puede fragmentar los bloques grandes que suelen ubicarse al final de la memoria.
----------	--	---	--

Como se observa en el análisis, no existe un algoritmo "mejor" de manera universal. La elección depende del patrón de solicitudes de memoria y los objetivos de rendimiento. Si la velocidad de asignación es crítica, First-Fit o Next-Fit son preferibles. Si se busca optimizar el aprovechamiento estricto del espacio en cada partición (y si los tiempos de búsqueda no son un cuello de botella), Best-Fit puede ser más adecuado. Sin embargo, todos sufren de fragmentación externa bajo particionamiento dinámico.

5. Conclusiones

- Se ha comprobado que la gestión de memoria dinámica es susceptible al problema de la fragmentación externa, como se evidenció de manera práctica en la imposibilidad de asignar el proceso de tamaño 301 en First-Fit y Next-Fit, a pesar de existir memoria libre total suficiente.
- La implementación evaluada para Best-Fit simplifica el particionamiento dinámico marcando los bloques como inutilizables tras su primera asignación, aproximándose más a un comportamiento de partición fija. Esto destaca la importancia de revisar minuciosamente la lógica de los algoritmos de simulación en comparación con su contraparte teórica.
- La principal diferencia observada entre First-Fit y Next-Fit radica en la retención del puntero de búsqueda. Aunque en el escenario simulado los resultados en términos de satisfacción de procesos fueron similares, Next-Fit demostró su cualidad teórica de avanzar a través del espacio de memoria evitando el re-escaneo constante de los primeros bloques.
- La estrategia de asignación a emplear debe ser elegida en base a las cargas de trabajo esperadas. Ningún algoritmo es inmune a la fragmentación ni ideal para cualquier caso de uso.

6. Referencias Bibliográficas

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). Wiley.

TutorialsPoint. (s.f.). *Operating System - Best Fit Allocation*. Recuperado de https://www.tutorialspoint.com/operating_system/os_best_fit_algorithm.htm

TutorialsPoint. (s.f.). *Operating System - Next Fit Allocation*. Recuperado de https://www.tutorialspoint.com/operating_system/os_next_fit_algorithm.htm